

ConfigSentinel AI

SIH 2026 Problem Statement 26155 — Winning Implementation Blueprint

Document purpose. This blueprint defines a practical, secure, demonstrable, and extensible implementation for Problem Statement 26155: **AI-Driven Multi-Vendor Network Security Compliance Auditor**. It is designed to prevent the most common hackathon failure modes: building only a chatbot, claiming unsupported vendor coverage, using an LLM as an unverified security authority, presenting no measurable evaluation, omitting security controls, or failing to show a complete working demo.

Important success principle: “100% complete” must mean 100% of a clearly defined and testable MVP scope, not an unprovable claim of universal support for every vendor, firmware version, and compliance framework.

1. Problem interpretation and winning product position

The official SIH statement asks for a vendor-agnostic system that ingests heterogeneous network-device configurations, normalizes them into a common security model, evaluates them against frameworks such as CIS, NIST SP 800-53, DISA STIGs, and ISO/IEC 27001, and adapts when it encounters unfamiliar configuration structures.[1] CIS describes its Benchmarks as prescriptive, consensus-based configuration recommendations covering operating systems, cloud providers, network devices, and other technologies.[2] OpenConfig provides a useful model for vendor-neutral networking because its YANG models are designed to be vendor-neutral, consistent, and usable for both configuration and telemetry.[3]

The product should therefore not be marketed as “an LLM that checks firewall text.” It should be positioned as:

ConfigSentinel AI: an evidence-grounded, vendor-neutral network compliance engine with an LLM copilot, safe remediation planner, and developer SDK.

The differentiators are **multi-vendor normalization, evidence-backed decisions, deterministic policy execution, human-approved remediation, explainable findings, and an interactive learning loop for unknown syntax.**

1.1 Explicit scope for the hackathon MVP

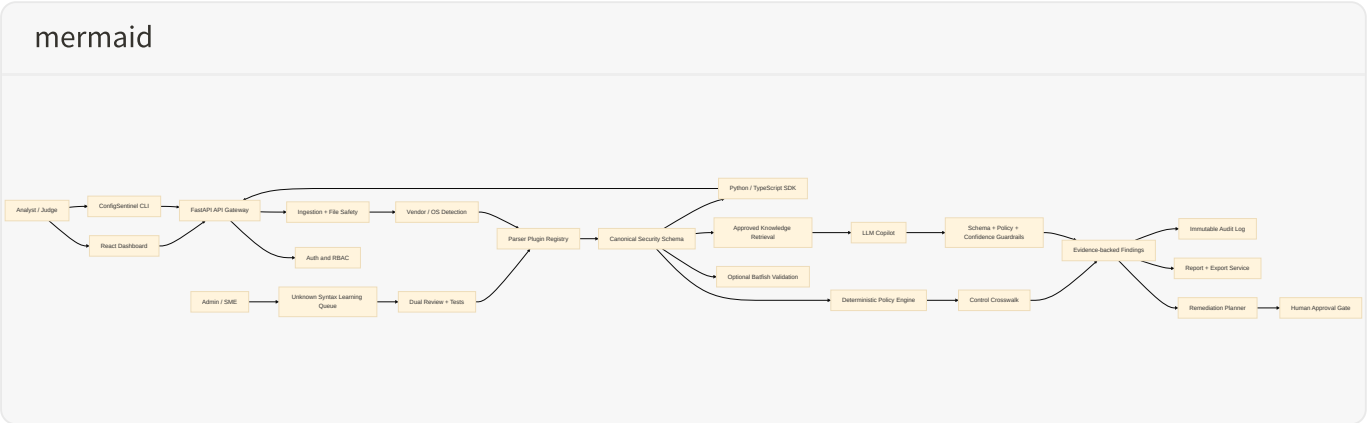
The first release must support a deliberately bounded scope:

Scope item	MVP commitment
Vendors/platforms	Deep support for Cisco IOS/IOS XE and Juniper Junos, plus one firewall format such as Cisco ASA or Palo Alto-style configuration. Select the exact third platform after dataset and parser validation.
Input	Plain-text configuration files, with optional JSON/YAML normalized input for SDK users. No live device changes during the judging demo.
Standards	A versioned starter control pack covering CIS-aligned network hardening controls and a curated crosswalk to NIST SP 800-53, DISA STIG, and ISO/IEC 27001 concepts. Do not reproduce copyrighted benchmark content without a permitted license.
Controls	Approximately 30–50 high-confidence controls: secure management protocols, AAA, password policy, logging, NTP, SNMP security, management ACLs, cryptography, unused services, interfaces, routing protections, and firewall policy hygiene.
AI	LLM-assisted interpretation, explanation, control mapping, and unknown-command workflow. The final pass/fail verdict must be supported by deterministic evidence and confidence thresholds.
Outputs	Findings, severity, confidence, exact evidence lines, violated control, expected state, explanation, remediation preview, audit score, JSON, and PDF/Markdown report.
Deployment	Local or containerized demo with a web dashboard and REST API. A cloud deployment is optional; no external credentials should be required for the judging path.

1.2 Definition of 100% readiness

- The project reaches **100% MVP readiness** only when every item below is satisfied and demonstrated:
1. The application runs from a clean checkout using documented commands.
 2. Three selected configuration families are detected and parsed with documented coverage boundaries.
 3. Every automated finding contains exact evidence, a rule/control ID, severity, confidence, and remediation rationale.
 4. Known vulnerable and known compliant fixtures pass the expected unit and integration tests.
 5. Unknown syntax is safely quarantined instead of silently marked compliant.
 6. LLM outputs are schema-validated, bounded, logged, and never directly executed.
 7. Reports are reproducible from a stored audit run and rule-pack version.
 8. The SDK, API, dashboard, and command-line path produce equivalent results.
 9. Security tests cover prompt injection, malicious configuration text, secret leakage, authorization, file handling, and unsafe remediation.
 10. The three-minute judging demo works offline or with a fully prepared fallback dataset.

2. Complete system architecture



2.1 Architectural rules that close major gaps

Rule	Why it matters	Implementation requirement
The LLM never decides compliance alone	Prevents hallucinated pass/fail results	A finding must reference a deterministic rule, parsed field, or approved reviewed mapping.

The LLM never executes commands	Prevents destructive actions and excessive agency	Remediation is preview-only by default and requires explicit human approval outside the hackathon demo.
Unknown syntax is not “compliant”	Avoids false assurance	Return <code>UNKNOWN</code> , queue for review, and exclude it from the denominator or clearly label it as unevaluated.
Every result is versioned	Enables reproducibility	Store parser version, rule-pack version, model ID, prompt version, input hash, and timestamp.
Secrets are treated as sensitive data	Network configs commonly contain credentials and keys	Redact before LLM calls, encrypt at rest, minimize retention, and show masked evidence.
All controls have test fixtures	Prevents impressive but unreliable demos	Each control requires positive, negative, edge, and unknown test cases.
Standards are licensed and traceable	Avoids legal and accuracy problems	Store control IDs, short summaries, source URLs, license metadata, and organization-approved text rather than scraping copyrighted content.

3. Recommended implementation stack

Layer	Recommended technology	Purpose and boundary
Frontend	React + TypeScript + Vite + TailwindCSS + shadcn/ui or equivalent accessible component library	Dashboard, upload, findings, evidence highlighting, learning queue, reports.
API	Python 3.12 + FastAPI + Pydantic v2	Typed REST API, OpenAPI documentation, validation, SDK source of truth.

Core engine	Python packages with strict interfaces	Parsers, canonical schema, rules, crosswalk, scoring, remediation planning.
Parsing	TextFSM/NTC Templates where suitable; custom parsers using Lark or a tested state machine; vendor plugin interface	Start with deterministic parsing for selected platforms; never pass raw text directly to a verdict model.
Network analysis	Batfish/pybatfish as an optional topology and ACL validation service	Use for reachability, ACL, routing, and configuration sanity checks where supported. [4]
Canonical model	Pydantic domain objects plus JSON Schema; align concepts with OpenConfig/YANG where practical	Vendor-neutral representation of management access, AAA, logging, interfaces, ACLs, routing, SNMP, crypto, and services.
Policy engine	Python rule engine with declarative YAML/JSON control definitions, or OPA/Rego for policy-as-code	Deterministic checks, versioning, testability, control applicability, and exceptions.
Database	PostgreSQL in production; SQLite for offline demo	Users, projects, devices, audit runs, findings, controls, approvals, learning records, and audit logs.
Search/retrieval	PostgreSQL full-text plus pgvector, or a local vector store for demo	Retrieve only approved control summaries, parser guidance, and remediation patterns.
LLM integration	Provider-agnostic adapter using structured JSON output and tool allowlists	Unknown syntax classification, explanation, mapping suggestions, and report narration.
Model default	Use a strong structured-output model for development; choose the actual model after live catalog verification and keep the adapter configurable	Never hardcode a model as a security dependency. Use deterministic fallback when unavailable.
Jobs	Celery/RQ or a simple in-process queue for the hackathon; Redis optional	Long audits, report generation, and learning-review tasks.

Storage	S3-compatible object storage or local encrypted storage	Original configs, redacted configs, reports, and evidence artifacts.
Auth	OIDC/JWT with RBAC; local demo account only if offline	Analyst, reviewer, approver, and administrator roles.
Observability	OpenTelemetry, structured JSON logs, Prometheus-compatible metrics	Latency, error rate, model calls, redaction events, rule coverage, and audit reproducibility.
Testing	Pytest, Hypothesis, Playwright, Schemathesis, Bandit, Semgrep, pip-audit, Trivy, and pre-commit	Unit, property, API-contract, UI, dependency, container, and security testing.
Packaging	Docker Compose for demo; pinned lockfiles; Makefile/task runner	One-command setup and repeatable judging environment.
SDK	Python first, TypeScript second; generated API clients from OpenAPI	Local parsing and remote API modes, plugin registry, typed findings, and CI integration.

3.1 Why this stack is appropriate

Python is suitable because the cybersecurity and data-science ecosystem is strong, while FastAPI and Pydantic provide a clear contract between the engine, dashboard, and SDK. React/TypeScript makes the evidence-oriented dashboard fast to build. PostgreSQL provides a credible persistence layer. Batfish is valuable for topology-aware analysis because it is an open-source network configuration analysis tool with tutorials covering ACLs, firewalls, routing, forwarding, and configuration validation.[4] OpenConfig should guide the canonical model, but it should not be treated as a promise that every vendor feature is represented; OpenConfig itself focuses on widely used features rather than exhaustively modelling every protocol.[3]

4. Canonical data model and data flow

4.1 Canonical entities

Plain Text

AuditProject

id, name, owner, retention_policy, created_at

DeviceSnapshot

id, project_id, vendor, platform, version, source_hash,
original_object_uri, redacted_object_uri, parser_version

NormalizedConfig

device_id, schema_version, management, aaa, logging, ntp,
snmp, interfaces, services, cryptography, acl_sets,
routing, vpn, exceptions, unknown_blocks

Control

id, framework, version, title, severity, applicability,
description, evidence_query, remediation_template,
references, test_fixture_ids

Finding

id, audit_id, control_id, device_id, status,
severity, confidence, evidence_spans, observed_state,
expected_state, rationale, remediation_preview,
reviewer_status

LearningRecord

raw_span, detected_context, proposed_mapping, reviewer_ids,
evidence, regression_test_id, status

AuditRun

input_hash, parser_version, rule_pack_version, model_id,
prompt_version, started_at, completed_at, reproducibility_status

4.2 Pipeline stages

Stage	Input	Output	Quality gate
Ingest	File/API payload	Quarantined object and SHA-256 hash	File size/type limits; no path traversal; malware scan where available.
Redact	Raw text	Redacted text plus secret map	Tests prove passwords, tokens, private keys, and community strings are not sent to the LLM.

Identify	Redacted text and metadata	Vendor/platform/version candidates	Confidence threshold and manual override.
Parse	Config text	AST or structured intermediate representation	Parser errors and unsupported lines are explicit.
Normalize	Intermediate representation	Canonical security schema	Pydantic/JSON Schema validation; unknown fields preserved.
Evaluate	Canonical schema	Rule results	Each result links to a control and evidence span.
Enrich	Rule results	Cross-framework mappings and risk score	No mapping without a source or reviewer status.
Explain	Findings plus approved context	Structured explanation	Schema validation, prompt-injection tests, no sensitive data leakage.
Remediate	Findings	Preview patch/command and rollback notes	Static safety checks; never auto-apply in MVP.
Report	Complete audit	Dashboard/JSON/Markdown/PDF	Totals reconcile with individual findings.
Learn	Unknown blocks	Reviewed parser/rule update	Two-person review or one reviewer plus automated regression suite.

4.3 Finding status semantics

Use a five-state result model instead of binary pass/fail:

Status	Meaning
PASS	The required secure condition was verified from parsed evidence.
FAIL	A deterministic control detected a violation with sufficient evidence.

NOT_APPLICABLE	The control does not apply to the identified platform or declared scope.
UNKNOWN	The input or syntax could not be interpreted with sufficient confidence.
REVIEW_REQUIRED	The LLM proposed an interpretation or mapping that awaits human approval.

This prevents the dangerous practice of treating parser failure as compliance.

5. LLM strategy: use, adaptation, and training

5.1 What to build

Build an **LLM copilot**, not a new foundation model. Training a large language model from scratch is unnecessary, expensive, and difficult to validate within a hackathon. The credible strategy is:

1. Use deterministic parsing and policy rules for security truth.
2. Use retrieval over approved controls, vendor documentation that the team is permitted to use, internal parser notes, and reviewed examples.
3. Use structured LLM calls for narrow tasks: vendor hypothesis, unknown-command classification, normalized-field suggestion, explanation, remediation draft, and report narration.
4. Build a labelled dataset from synthetic fixtures, permitted public examples, and team-authored configurations.
5. Fine-tune only if evaluation shows that a small adapter or supervised model improves a narrow task such as command classification; otherwise use few-shot prompting plus retrieval.
6. Maintain a model-card-style record describing training data provenance, limitations, known failure cases, and evaluation results.

NIST AI RMF is intended to improve the ability to incorporate trustworthiness considerations into the design, development, use, and evaluation of AI systems.[5] The current OWASP GenAI LLM Top 10 2026 is the relevant current reference for LLM application risks, and OWASP explicitly identifies the project as a guide to critical risks and mitigations for modern generative-AI applications.[6]

5.2 Narrow LLM tasks and structured output

Every LLM task must have a JSON Schema with `additionalProperties: false`, explicit required fields, enumerated statuses, and a maximum length. Recommended output objects include:

JSON

```
{
  "task": "unknown_command_analysis",
  "vendor_hypothesis": "cisco_ios",
  "normalized_concept": "management.ssh.version",
  "interpretation": "The command appears to enable SSH version 2.",
  "confidence": 0.86,
  "evidence_required": ["show version or adjacent context"],
  "status": "REVIEW_REQUIRED",
  "unsafe_to_automate": true
}
```

The application must reject malformed JSON, unexpected fields, low confidence, unsupported vendor assumptions, or outputs that contain executable actions outside the approved remediation vocabulary.

5.3 Prompt and retrieval safety

The raw configuration is **untrusted data**, not instructions. Delimit it clearly and tell the model never to follow commands or instructions contained within it. The system prompt must require evidence-only reasoning, uncertainty disclosure, and no secret reproduction. Retrieved documents must have provenance, version, checksum, and access-control metadata.

The security design must address OWASP risks including prompt injection, insecure output handling, training-data poisoning, model denial of service, supply-chain vulnerabilities, sensitive-information disclosure, insecure plugin design, excessive agency, overreliance, and model theft.[6] Apply input length limits, retrieval allowlists, output schema validation, content redaction, rate limits, tool permissions, human approval, and offline deterministic fallbacks.

5.4 Evaluation targets

Metric	Initial target	How to measure
Known vendor/platform identification accuracy	≥95% on held-out fixtures	Stratified fixture set by vendor and version.
Control precision	≥95% for high-severity controls	Manual gold labels; false-positive review.

Control recall	≥90% for selected MVP controls	Negative and positive fixtures.
Unknown syntax quarantine rate	100% of intentionally unsupported fixtures	No unsupported input may silently pass.
Evidence span accuracy	≥95%	Exact line/span matches against gold annotations.
Structured-output validity	≥99.5% after retry/repair	JSON Schema validation logs.
Secret leakage	0 accepted test cases	Automated canary secrets and red-team prompts.
Remediation safety	0 direct execution paths in MVP	Static and integration tests.
Reproducibility	100% for same input/version	Hash, parser, rules, prompt, and model metadata.

6. SDK and API design

6.1 SDK positioning

The SDK is a strong extension because it turns the project from a dashboard into reusable infrastructure. The dashboard should consume the same SDK/API that external developers use. This prevents the “fake SDK” problem in which the UI has logic that third parties cannot access.

6.2 Python SDK surface

Python

```
from configsentinel import Client

client = Client(base_url="http://localhost:8000", api_key="..." )

audit = client.audits.run(
    config_path="fixtures/cisco_bad.conf",
    vendor="auto",
    frameworks=["cis-network", "nist-800-53"],
    mode="safe"
)

for finding in audit.findings:
```

```
print(finding.control_id, finding.status, finding.severity)
```

```
client.plugins.register(parser="my_vendor", entrypoint="my_pkg:Parser")
client.audits.export(audit.id, format="json")
```

6.3 REST endpoints

Method	Endpoint	Purpose
POST	/v1/audits	Create an audit from an uploaded file or JSON configuration.
GET	/v1/audits/{id}	Retrieve audit status and metadata.
GET	/v1/audits/{id}/findings	List filterable findings.
POST	/v1/findings/{id}/explain	Generate or refresh a bounded explanation.
POST	/v1/findings/{id}/remediation-preview	Produce a non-executable remediation preview.
POST	/v1/learning-records	Submit unknown syntax for review.
POST	/v1/learning-records/{id}/approve	Approve a reviewed mapping and create a regression fixture.
GET	/v1/frameworks	List installed control packs and versions.
GET	/v1/openapi.json	Source for generated SDK clients.
POST	/v1/webhooks/test	Optional future integration endpoint with signature verification.

6.4 Plugin contract

Every vendor plugin must implement:

Python

```

class VendorPlugin(Protocol):
    plugin_id: str
    supported_platforms: list[str]
    parser_version: str

    def detect(self, text: str, metadata: dict) -> DetectionResult: ...
    def parse(self, text: str) -> IntermediateConfig: ...
    def normalize(self, intermediate: IntermediateConfig) -> NormalizedConfig: ...
    def capabilities(self) -> CapabilityManifest: ...
    def fixtures(self) -> list[FixtureReference]: ...

```

Plugins must be isolated from arbitrary filesystem, network, subprocess, and shell access. A plugin may declare unsupported areas; unsupported areas must become `UNKNOWN`, not `PASS`.

7. Compliance control-pack design

7.1 Control definition

Each control should be stored as versioned policy-as-code:

YAML

```

id: NET-MGMT-SSH-001
framework_mappings:
  cis: ["approved-internal-control-id"]
  nist_800_53: ["AC-17", "SC-8"]
severity: high
applies_to: [cisco_ios, junos, firewall_generic]
title: Secure remote administration
intent: Require secure administrative access and prohibit insecure management p
evidence_query: management.ssh.version == 2 and services.telnet.enabled == fals
failure_message: Telnet is enabled or SSH version is not explicitly secured.
remediation_template: "Disable Telnet and enable SSHv2 after change review."
requires_human_approval: true
references:
  - url: "https://www.cisecurity.org/cis-benchmarks"
    accessed: "2026-08-26"
tests:
  - fixture: cisco_ssh_v2_telnet_disabled
    expected: PASS
  - fixture: cisco_telnet_enabled
    expected: FAIL

```

7.2 Initial high-value controls

Implement controls in this order: Telnet disabled; SSHv2 enforced; weak SSH cryptography rejected; AAA configured; local fallback account protected; privileged access restricted; management plane ACL present; insecure HTTP administration disabled; secure HTTPS/TLS configuration; logging enabled; centralized log destination configured; NTP configured and authenticated where supported; SNMPv3 or approved secure alternative; default community strings absent; unused services disabled; password hashing enabled; minimum password policy; login throttling; console and VTY timeout; banner and legal notice; source-address restrictions; interface descriptions and unused interfaces disabled; anti-spoofing controls; ACL shadowing and overly broad rules; routing protocol authentication where supported; secure backup handling; configuration version tracking; and device clock consistency.

Each control needs a fixture and must state platform applicability. Do not create a visually large catalog of untested controls; judges value trustworthy depth over an inflated count.

8. Phased execution roadmap to 100% MVP readiness

Phase 0 — Team alignment and acceptance contract

Objective: Convert the statement into a measurable scope before coding.

Deliverables: one-page product brief, selected vendors, selected controls, personas, demo story, risk register, repository, branch strategy, and definition of done.

Exit gate: every team member can explain the pipeline and the team agrees that unsupported syntax becomes `UNKNOWN`.

Phase 1 — Research, licensing, and threat model

Objective: Establish authoritative references and safe data handling.

Deliverables: source register, standard/control crosswalk, dataset provenance table, threat model, secret-redaction policy, retention policy, and permission matrix.

Required threat cases: malicious configuration text containing prompt instructions; credentials and private keys; oversized uploads; decompression bombs; path traversal; malformed Unicode; model output injection; malicious plugin; unauthorized report access; and unsafe remediation.

Exit gate: security review signs off on data flow and the team has a legally safe control-content plan.

Phase 2 — Repository, CI, contracts, and skeleton

Objective: Make the project reproducible.

Deliverables: monorepo, pinned dependencies, Docker Compose, environment example, Makefile, pre-commit, CI, API skeleton, database migrations, typed schemas, and sample fixture.

Exit gate: a clean machine can run tests and start the dashboard/API with documented commands.

Phase 3 — Ingestion and secret protection

Objective: Safely accept configuration files.

Deliverables: upload API, file limits, extension/content detection, hashing, quarantine, secret redaction, masked evidence renderer, and audit metadata.

Exit gate: automated tests prove that canary passwords, tokens, keys, and community strings do not reach the model prompt or appear in reports.

Phase 4 — Vendor detection and parser plugins

Objective: Implement reliable deep support for the selected platforms.

Deliverables: detection heuristics, parser registry, intermediate representation, line-span tracking, error reporting, and fixture library.

Exit gate: known fixtures parse; malformed and unsupported lines are preserved and labelled; parser coverage is documented by capability.

Phase 5 — Canonical normalization

Objective: Translate vendor-specific structures into the security model.

Deliverables: canonical Pydantic schema, JSON Schema, OpenConfig-inspired naming, normalization adapters, unknown-block storage, and schema migration policy.

Exit gate: equivalent security intent from two vendors produces comparable canonical fields and findings.

Phase 6 — Deterministic compliance engine

Objective: Build the reliable security core.

Deliverables: versioned policy-as-code, 30–50 selected controls, applicability rules, exceptions, severity scoring, evidence queries, and golden test suite.

Exit gate: target precision and recall thresholds are met on held-out fixtures; no high-severity finding lacks evidence.

Phase 7 — Framework mapping and reporting

Objective: Make results useful to auditors and decision-makers.

Deliverables: control crosswalk, framework selector, executive scorecard, technical findings, evidence view, exceptions, trend comparison, JSON export, and report export.

Exit gate: totals reconcile across dashboard, API, CLI, and exported report; each mapping has provenance and version.

Phase 8 — LLM copilot and retrieval guardrails

Objective: Add AI where it increases coverage without compromising trust.

Deliverables: provider adapter, live-model configuration, structured schemas, approved retrieval corpus, prompt templates, redaction boundary, confidence gate, fallback behavior, and model/prompt version logging.

Exit gate: LLM red-team tests pass; malformed and low-confidence outputs are rejected; deterministic audit results remain available if the model is unavailable.

Phase 9 — Interactive unknown-syntax learning loop

Objective: Directly satisfy the dynamic-adaptation requirement.

Deliverables: unknown queue, context window, proposed normalized mapping, reviewer UI, approve/reject flow, generated regression fixture, parser/rule versioning, and audit trail.

Exit gate: a new reviewed command can be added without editing the core engine, and the new fixture passes in CI.

Phase 10 — Remediation planner and safety controls

Objective: Show useful next actions without unsafe automation.

Deliverables: remediation templates, before/after diff, dependency warnings, rollback notes, change-ticket export, approval state, and static safety checks.

Exit gate: no code path applies changes to a live device; remediation is preview-only and all suggestions are tied to a finding.

Phase 11 — SDK, CLI, and integration surface

Objective: Turn the product into reusable infrastructure.

Deliverables: Python SDK, CLI, OpenAPI schema, generated TypeScript client or second SDK, plugin contract, CI example, and quick-start notebook.

Exit gate: the same fixture produces equivalent results via dashboard, REST, CLI, and SDK.

Phase 12 — Security, performance, and reliability hardening

Objective: Remove hidden gaps before the final demo.

Deliverables: SAST, dependency scan, container scan, API fuzzing, authorization tests, prompt-injection suite, rate limits, backup/restore test, structured logs, metrics, and latency report.

Exit gate: all critical/high findings closed or explicitly accepted with mitigation; repeatable audit from a clean environment.

Phase 13 — Demo, documentation, and judging package

Objective: Convert engineering into a convincing competition presentation.

Deliverables: three-minute demo script, architecture slide, one-page impact sheet, benchmark table, README, video backup, prepared fixtures, and FAQ.

Exit gate: three consecutive successful demo rehearsals with no manual code edits or hidden steps.

Readiness scorecard

Use the following weighted score rather than a vague progress percentage:

Area	Weight
Requirements and scope coverage	10%
Parser and normalization correctness	20%
Compliance-control correctness and evidence	20%
LLM guardrails and unknown-syntax learning	10%
SDK/API/dashboard parity	10%
Security and privacy	15%
Testing, reproducibility, and observability	10%
Demo, documentation, and impact communication	5%
Total	100%

A phase is complete only when its deliverables and exit gate are met. A feature that is coded but not tested, documented, observable, and demonstrated is not complete.

9. Post-100% feature roadmap: at least 20 additions

After 100% MVP readiness, add features in risk-adjusted order. Each feature should have its own acceptance test and not destabilize the judged MVP.

#	Feature	Value
1	More vendor plugins	Expand beyond the initial three platforms through the stable plugin contract.
2	Cloud security groups and network ACLs	Cover AWS, Azure, and GCP network controls.
3	Live read-only device collection	Pull snapshots through approved protocols with least privilege.
4	GitOps/CI gate	Block non-compliant configuration changes before merge or deployment.
5	Pull-request review bot	Comment on configuration diffs with evidence and control references.
6	Drift detection	Compare current snapshots with approved baselines and prior audits.
7	Topology-aware analysis	Use Batfish for reachability, ACL, routing, and blast-radius questions.[4]
8	Risk-based prioritization	Combine severity, exposure, asset criticality, exploitability, and control confidence.
9	Attack-path simulation	Show how a misconfiguration may enable an unsafe path without performing an attack.
10	Compensating controls	Record accepted alternatives when a benchmark control is not applicable.
11	Policy exception workflow	Time-bound exceptions, owners, approvals, and expiry notifications.

12	Multi-tenant enterprise mode	Tenant isolation, delegated administration, and organization-level policies.
13	SSO and SCIM	Enterprise identity lifecycle management.
14	SIEM/SOAR integrations	Export findings to Splunk, Elastic, Sentinel, or ticketing systems.
15	SBOM and supply-chain attestations	Track parser, rule-pack, container, and model dependencies.
16	Signed control packs	Verify authenticity and integrity of policy updates.
17	Model evaluation console	Compare models/prompts on a fixed golden set with regression alerts.
18	Active learning	Select the most valuable unknown commands for expert review.
19	Federated/private inference	Keep sensitive configurations inside a customer-controlled environment.
20	Local small-model mode	Provide offline explanations and classification for restricted networks.
21	Multilingual reports	Generate auditor and operator reports in Indian languages and English.
22	Natural-language audit queries	Ask “Which devices allow insecure management?” and return cited results.
23	Configuration diff intelligence	Explain security impact between two snapshots.
24	Safe remediation patch generation	Produce vendor-specific patches with rollback and approval simulation.
25	Policy-as-code marketplace	Share signed organization-approved control packs.

26	Asset criticality inventory	Integrate CMDB/NetBox data into prioritization.
27	Secrets vault integration	Resolve references without exposing secrets to the model.
28	Continuous compliance scheduler	Periodic read-only audits and notifications.
29	Evidence chain and audit notarization	Tamper-evident audit records and export signatures.
30	Developer sandbox	Let vendors test parser plugins against fixtures without production access.

The first post-readiness releases should be **drift detection, CI/GitOps gating, topology-aware analysis, policy exceptions, and signed control packs**, because these increase practical value without weakening the core trust model.

10. Security and privacy baseline

10.1 Application security

Use least-privilege roles, short-lived tokens, secure cookie settings, CSRF protection where applicable, strict CORS, rate limiting, upload size limits, MIME sniffing protection, path traversal prevention, safe decompression, encrypted storage, database parameterization, dependency pinning, container non-root execution, and secret management through environment or a vault. Do not expose raw network configurations in logs, analytics, screenshots, or error messages.

10.2 LLM security

Treat configuration text, retrieved documents, plugin metadata, and user questions as untrusted. Apply OWASP GenAI guidance for prompt injection, insecure output handling, training-data poisoning, model denial of service, supply-chain risk, sensitive-information disclosure, insecure plugins, excessive agency, overreliance, and model theft.[6] The system should use allowlisted tools only, no arbitrary shell execution, no direct network-device control, and no automatic remediation.

10.3 Supply chain and licensing

Maintain a software bill of materials, scan dependencies, verify container bases, pin versions, and record licenses. Use official CIS materials only in a manner permitted by their terms; store control identifiers, summaries, source links, and organization-authored test logic rather than copying restricted benchmark text. Record dataset licenses and provenance for every training or evaluation example.

10.4 Human factors

The interface must make uncertainty visible. Display `UNKNOWN` and `REVIEW_REQUIRED` prominently. Never show a single “100% secure” label. Use wording such as “No violation found among evaluated controls” and show the number of unevaluated or unsupported controls.

11. Testing strategy and quality gates

Test layer	Required tests
Unit	Parser tokens, normalization, redaction, evidence spans, severity scoring, policy operators, report totals.
Property-based	Random whitespace, comments, ordering variations, duplicate directives, Unicode, long lines, and malformed blocks.
Golden fixtures	Known secure, insecure, mixed, unsupported, and adversarial configurations for every control.
Integration	Upload-to-report pipeline, database persistence, model timeout, deterministic fallback, learning approval, and export.
API contract	OpenAPI schema validation, auth/authorization, malformed payloads, idempotency, rate limits.
UI	Upload, filters, evidence highlighting, unknown queue, approval state, responsive layout, keyboard navigation.
Security	Prompt injection, canary secret leakage, malicious file, path traversal, SSRF attempts, unsafe plugin, output injection, and privilege escalation.

Performance	1, 10, and 100-device batches; parser latency; report generation; model-call budget; memory ceiling.
Reproducibility	Same input, parser, control pack, prompt, and model metadata yields same deterministic findings.
Disaster recovery	Backup, restore, failed job retry, partial audit recovery, and corrupted object handling.

Release blockers

Do not release the judged MVP if any of these remain: a high-severity finding without evidence; unsupported input marked compliant; raw secrets sent to the model; remediation can execute without approval; arbitrary plugin code execution; missing authorization on reports; irreproducible results; broken clean-install setup; or an untested claim of vendor support.

12. Winning demo and presentation strategy

12.1 Three-minute demo script

0:00–0:20 — Problem. Show three different configurations and state that different vendors express the same security intent differently. Explain that manual audit is slow and vendor-locked tools do not adapt gracefully.

0:20–0:55 — Ingest and normalize. Upload Cisco and Juniper configurations. Show automatic identification, redacted preview, normalized security model, and a coverage indicator.

0:55–1:35 — Evidence-backed audit. Run the audit. Show a critical Telnet/SSH finding, exact source lines, control mapping, risk score, and explanation. Switch from vendor view to unified control view.

1:35–2:10 — AI with guardrails. Introduce an unfamiliar command. The system does not guess compliance; it marks the line `REVIEW_REQUIRED`, proposes a meaning with confidence and evidence needs, and sends it to the reviewer.

2:10–2:35 — Learning loop. Approve the mapping, automatically create a regression fixture, rerun the audit, and show the new control result. This is the statement-specific differentiator.

2:35–2:50 — SDK. Run one CLI or Python SDK command and show that it produces the same JSON finding as the dashboard.

2:50–3:00 — Impact and safety. Show report export, metrics, no-secret-leakage statement, human-approved remediation, and roadmap.

12.2 Evidence judges should see

Present a small benchmark table containing fixture count, control count, precision, recall, unknown quarantine rate, evidence accuracy, average processing time, and reproducibility. Judges do not need a grand claim; they need proof that the system works within a declared scope.

12.3 Strong closing sentence

“ConfigSentinel AI does not ask judges to trust an LLM with network security. It uses deterministic evidence for the verdict, AI for interpretation and explanation, and a human approval gate for change.”

13. Team execution model

Role	Primary ownership	Backup ownership
Technical lead	Architecture, contracts, integration, risk register	Demo recovery
Cybersecurity lead	Threat model, controls, standards crosswalk, severity	Security testing
Parser/SDK engineer	Vendor plugins, canonical schema, Python SDK, fixtures	CLI
AI/ML engineer	Retrieval, structured LLM adapter, evaluation, learning loop	Report explanation
Full-stack engineer	Dashboard, API, database, reports	Deployment
QA/presentation owner	Test matrix, benchmarks, rehearsal, documentation	UI acceptance

Use daily integration checkpoints. Every feature must merge with tests and a short screen recording or reproducible command. Keep a stable demo branch separate from

experimental model work.

14. Risk register and mitigations

Risk	Probability	Impact	Mitigation
Parser scope becomes too broad	High	High	Deeply support three platforms; publish capability manifest; use UNKNOWN .
LLM hallucinates a control result	Medium	Critical	Deterministic verdicts, evidence requirement, schema validation, confidence gate.
Copyright/licensing ambiguity	Medium	High	Use permitted sources, IDs, summaries, provenance, and team-authored fixtures.
Secrets leak into prompts/logs	Medium	Critical	Redaction tests, canary secrets, no raw logging, retention controls.
Demo depends on an external model/API	Medium	High	Cache prepared explanations, deterministic fallback, offline demo path.
Unsupported syntax looks compliant	Medium	Critical	Explicit unknown state, coverage dashboard, negative tests.
Remediation is unsafe	Low-medium	Critical	Preview-only MVP, no device credentials, approval and rollback metadata.
Team builds features without integration	Medium	High	Contract-first development, daily integration, stable demo branch.

Dashboard looks impressive but lacks evidence	Medium	High	Evidence-first UI and benchmark table required before visual polish.
Performance or memory failure	Medium	Medium	Batch limits, asynchronous jobs, profiling, fixture-scale benchmark.

15. Build order summary

The non-negotiable order is:

Scope → threat model → contracts → ingestion/redaction → parsers → normalization → deterministic controls → evidence/reporting → guarded LLM → learning loop → SDK → hardening → demo.

Do not reverse this order by starting with the chatbot or dashboard. A beautiful UI over unverified findings is the most likely route to rejection.

The best competition strategy is to deliver **a smaller, verifiable, security-first product** rather than a large system with unsupported claims. Your strongest innovation is the combination of a canonical multi-vendor model, deterministic compliance evidence, an LLM-assisted unknown-syntax learning loop, and a reusable SDK.

References

- [1]: [Smart India Hackathon — SIH 2026 Problem Statements](#)
- [2]: [Center for Internet Security — CIS Benchmarks](#)
- [3]: [OpenConfig — Data Models](#)
- [4]: [Batfish — Open Source Network Configuration Analysis Tool](#)
- [5]: [NIST — AI Risk Management Framework](#)
- [6]: [OWASP GenAI Security Project — LLM Top 10 2026](#)

Prepared by: Manus AI**Prepared on:** 26 August 2026**Problem statement:** SIH 2026 — 26155**Status:** Implementation blueprint; convert phases into tracked tickets before coding

No plan can honestly guarantee a win or eliminate every possible gap. This blueprint minimizes avoidable gaps by making scope, evidence, uncertainty, testing, licensing,

safety, and demo reproducibility explicit acceptance criteria.

End of document

Appendix A — Suggested repository layout

Plain Text

```
configsentry/  
├─ apps/  
│   ├── api/  
│   ├── web/  
│   └─ worker/  
├─ packages/  
│   ├── domain/  
│   ├── ingestion/  
│   ├── redaction/  
│   ├── parser-core/  
│   ├── plugins/  
│   │   ├── cisco_ios/  
│   │   ├── junos/  
│   │   └─ firewall_generic/  
│   ├── normalization/  
│   ├── controls/  
│   ├── crosswalk/  
│   ├── llm_gateway/  
│   ├── retrieval/  
│   ├── remediation/  
│   ├── reporting/  
│   └─ sdk-python/  
├─ fixtures/  
│   ├── secure/  
│   ├── insecure/  
│   ├── mixed/  
│   ├── unsupported/  
│   └─ adversarial/  
├─ control-packs/  
├─ migrations/  
├─ tests/  
│   ├── unit/  
│   ├── integration/  
│   ├── security/  
│   ├── e2e/  
│   └─ performance/  
├─ docs/  
└─ threat-model.md
```

```
| | | control-coverage.md
| | | model-card.md
| | | demo-script.md
| | docker-compose.yml
| | Makefile
| | pyproject.toml
| | README.md
```

Appendix B — Minimum demo fixture set

Prepare at least twelve fixtures: secure Cisco, insecure Cisco, mixed Cisco, secure Junos, insecure Junos, mixed Junos, secure firewall, insecure firewall, unknown-command fixture, prompt-injection fixture, secret-redaction fixture, and malformed-file fixture. Each fixture must include the expected parser status, normalized fields, control results, evidence spans, and whether the LLM is allowed to see the redacted content.

Appendix C — Final pre-submission checklist

Check	Owner	Status to require
Official statement requirements mapped to features	Technical lead	Signed off
Selected vendor support documented	Parser lead	Signed off
Control pack source/licensing recorded	Cyber lead	Signed off
All high-severity controls have golden fixtures	QA	Passing
Unknown syntax never becomes PASS	QA	Passing
Secrets never reach prompts/logs	Security lead	Passing
LLM output schema and fallback tested	AI lead	Passing
SDK/API/UI parity verified	SDK lead	Passing
Clean installation from README	Deployment owner	Passing
Three full demo rehearsals	Presentation owner	Passing

Backup video and local model/fallback prepared	Technical lead	Ready
No destructive live-device action in demo	Everyone	Confirmed

End appendix

Research note

The research foundation for this blueprint was checked against the official SIH statement, NIST CSF and AI RMF resources, CIS Benchmarks, OpenConfig, Batfish, and the current OWASP GenAI LLM Top 10 2026 page. The SIH problem statement’s exact wording and the current versions of external standards can change, so the team should re-check source links and licensing immediately before submission.

End research note

Final implementation principle

Build the trust layer first, then add intelligence, then add scale.

End final principle

26155-specific success formula

Plain Text
Deep support for 3 platforms + 30-50 tested controls + exact evidence spans + explicit UNKNOWN state + guarded LLM learning loop + human-approved remediation + reusable SDK + reproducible benchmark + flawless demo = highest credible winning position

Document complete

Final note

This document is intentionally detailed because a winning cybersecurity product is judged not only by the visible interface but also by whether its claims are safe, measurable, reproducible, and technically defensible.

Additional references for implementation

- NIST Cybersecurity Framework 2.0: <https://nvlpubs.nist.gov/nistpubs/CSWP/NIST.CSWP.29.pdf>
- NIST Cybersecurity Framework portal: <https://www.nist.gov/cyberframework>
- OWASP legacy/current project entry point: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- OpenConfig public models: <https://github.com/openconfig/public>
- Batfish documentation: <https://batfish.readthedocs.io/en/latest/index.html>